PaleoPal – High-Level Architecture Overview

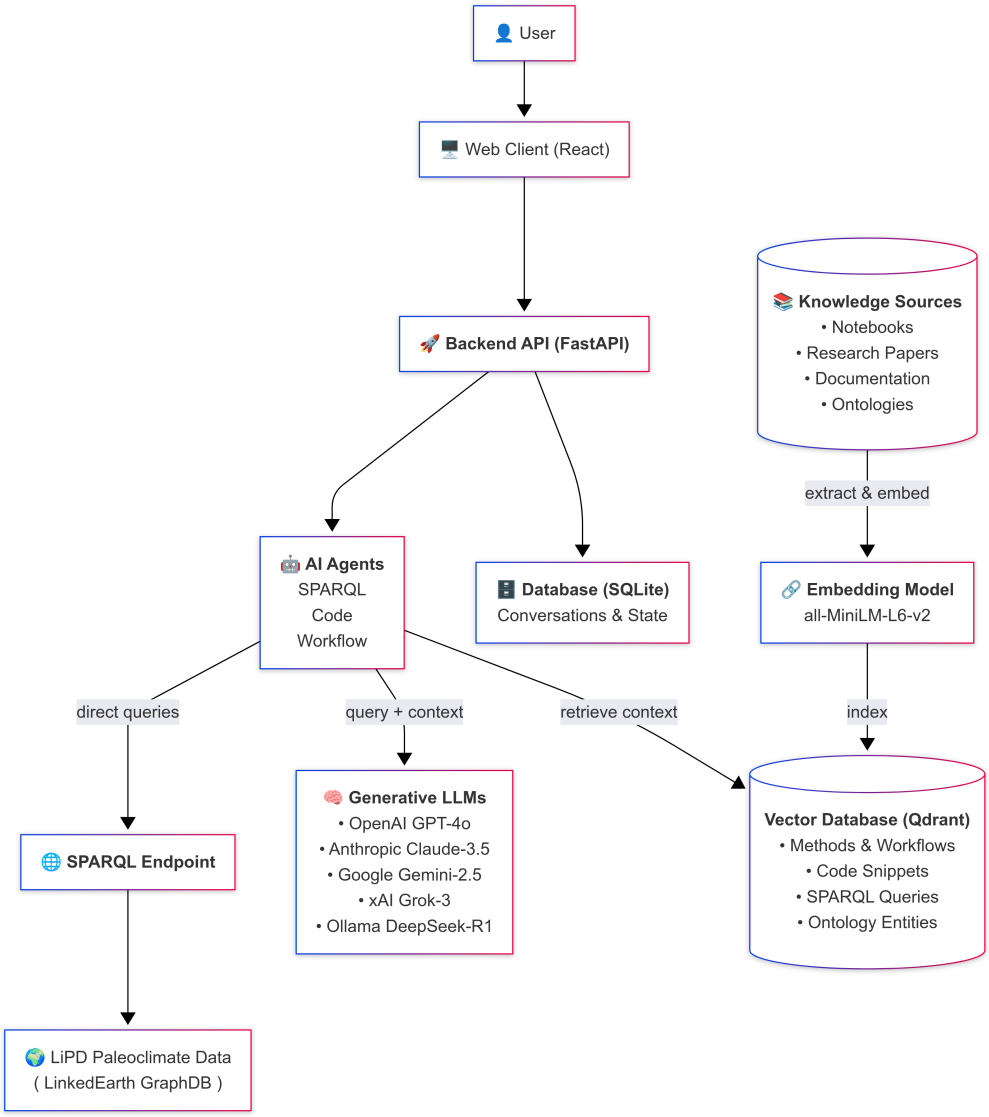

Diagram source: `ArchitectureDiagram.mmd`

PaleoPal is an AI-powered assistant for paleoclimate research. At its core it combines **retrieval-augmented generation (RAG)**, a set of specialised AI agents, and a vector knowledge base derived from notebooks, papers and ontologies.

---

1. Building Blocks

Layer	Key Component(s)	Role
User Interface	React single-page app	Chat UI & progress visualisation
API Layer	FastAPI service	REST endpoints + WebSocket streaming

Layer	Key Component(s)	Role
Core Engine	Agent Registry & three LangGraph agents	Orchestrates requests – chooses agent, streams results
Generative AI	OpenAI GPT-4o, Anthropic Claude-3.5, Google Gemini-2.5, xAI Grok-3, Ollama DeepSeek-R1	Large-language-model reasoning & content generation
Vector Knowledge Base	Qdrant (all-MiniLM-L6-v2 embeddings)	Semantic search over domain knowledge
Knowledge Sources	Notebooks, research papers, API docs, ontologies	Raw material (extracted → embedded → stored)
Metadata Store	SQLite	Conversations & agent state
External Data	LinkedEarth GraphDB SPARQL endpoint	Live paleoclimate datasets

2. Key Data Flow (RAG)

1. **User query** → UI → FastAPI.
2. **Agent selection** – registry routes the request to:
   - SPARQL Agent (build queries)
   - Code Agent (write analysis code)
   - Workflow Agent (plan multi-step workflows)
3. **Agent execution flow (summary):**
   - **SPARQL Agent** – similar-query lookup → entity match → (optional) clarification → query generation → execution → refinement.
   - **Code Agent** – example search → (optional) clarification → code generation → refinement.
   - **Workflow Agent** – request extraction → context search → (optional) clarification → workflow plan generation.
4. **Context retrieval** – agent performs semantic search against Qdrant **and gathers the running conversation history (previous code, queries, messages)**:
   - `sparql_queries`, `ontology_entities` → SPARQL Agent
   - `notebook_snippets`, `readthedocs_docs` → Code Agent
   - `literature_methods`, `notebook_snippets` → Workflow Agent
5. **Augmented prompt** – agent merges top-k docs with original question.
6. **Generation** – prompt sent to the selected LLM provider.
7. **Response streaming** back to the client.

---

3. Knowledge Ingestion Pipeline

```
Sources  ──►  Extractors  ──►  Embedding (all-MiniLM-L6-v2)  ──►  Qdrant
```

- **Notebooks** → code snippets & workflows (`notebook_snippets`, `literature_methods`)
- **Research papers** → method steps (`literature_methods`)
- **API documentation** → function/class snippets (`readthedocs_docs`)
- **Ontologies** → entities & relations (`ontology_entities`)
- **Notebooks / docs** → reusable SPARQL templates (`sparql_queries`)

Ingestion tasks are decoupled; new sources can be added without touching the main application.

---

4. Deployment Snapshot

- **Containers**: `frontend`, `backend`, `qdrant` (plus optional `graphdb`).
- **Internal network**: single Docker network for inter-service traffic.
- **Volumes**: persistent storage for Qdrant and SQLite.
- **Stateless services**: front & back ends can scale horizontally; state lives in Qdrant/SQLite.

---

5. Extensibility Hot-Spots

1. **Agents** – plug-in new LangGraph agents (e.g., visualisation agent).
2. **LLM providers** – switch/add via config/env vars.
3. **Collections** – create new Qdrant collections for fresh domains.
4. **Extractors** – contribute new document-type extractors.

Last updated 2025.

Detailed node-by-node execution flows for each agent are available in the full Technical Architecture paper (Section 5.2).